

# Creating and Deploying Production-Ready Agentic Systems

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center

June 27, 2026

# What we will be talking about

## 1 The LLM as a System Component

- Inherent LLM limitations
- What do LLMs bring to the table?
- When to use LLMs?

## 2 Tool-Based Reasoning using ReAct

- LLM tool-calling
- Thought, Action, Observation
- How much autonomy should we give the agent?

## 3 Agent-to-Agent Interaction

- Single-Agent vs. Multi-Agent
- Orchestrating Interaction
- What do we actually need?

# Section 1

## The LLM as a System Component

### Subsection 1

#### Inherent LLM limitations

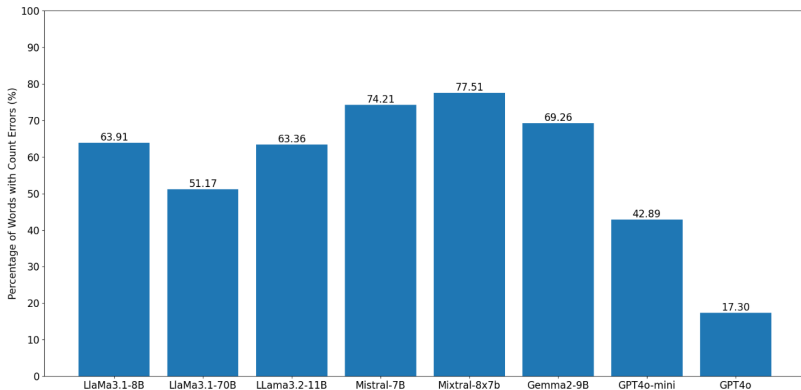
# Mathematics

## LLMs are really bad at math

- Predict the most likely sequence of tokens, not the logically necessary derivation.
- Probabilistic nature → prone to hallucinations.
- Tokenization absolutely destroys numbers.

While OK for small numbers (they can memorize some patterns), they generally fail at larger ones.

# Word counting



**Figure:** Percentage of words with errors on when counting letters for the different models  
<https://arxiv.org/html/2412.18626v1>.

# Character counting

## Example

```
model.prompt("How many 'r's are in 'carr'?")  
>>> "There is one 'r' in 'carr'".
```

## Indicative explanation

- The model can not find the word “carr” in its knowledge base...
- ...but it knows the word “car” and the fact that “carr” and “car” are highly correlated...
- ...“car” has one “r”, therefore this is the answer.

# Deductive reasoning

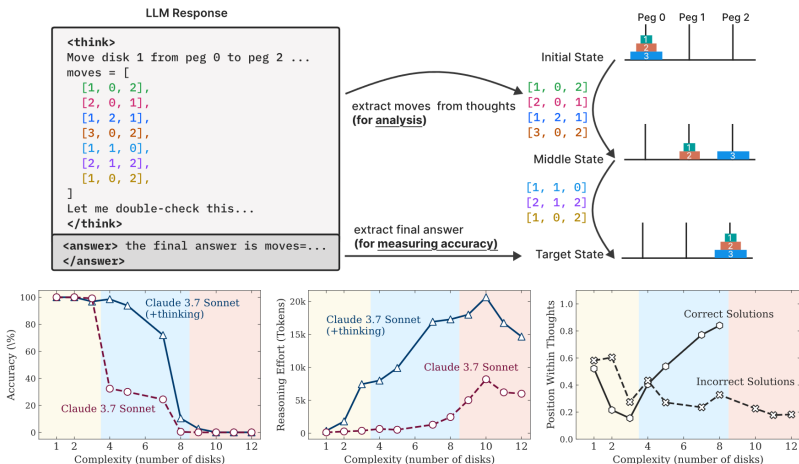


Figure: Percentage of words with errors on when counting letters for the different models

<https://arxiv.org/html/2412.18626v1>

# General Limitations

## LLMs cannot:

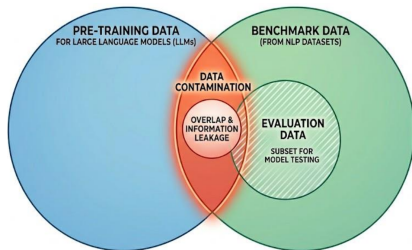
- Think.
- Behave.
- Deduct.

## Why?

- LLMs are generally **excellent stochastic parrots**. Their **next-word-prediction** nature prevents them from understanding the world and its mechanisms.
- Finetuning, reasoning and in-context learning alleviate, but do not bypass this restriction.



# A Note on Benchmarks

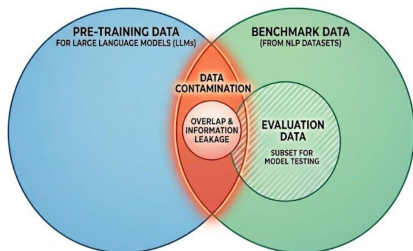


**Figure:** Credit to Electra Maria Papadopoulou—generated with Gemini.

## Data Contamination

- Evaluation data appears in training.
- Web-scraping may include datasets used as training, validation, or benchmarks, unknowingly inflating reported performance.
- Sometimes on purpose (“benchmark-maxxing”).

# A Note on Benchmarks



Behavior replication

LLMs

Figure: Credit to Electra Maria Papadopoulou—generated with Gemini.

# Section 1

## The LLM as a System Component

### Subsection 2

What do LLMs bring to the table?

# Section 1

## The LLM as a System Component

### Subsection 3

#### When to use LLMs?

# LLMs are not just task-specific tools

- So far, we have used LLMs as specialized tools for specific tasks.
- While useful, these tasks can generally be accomplished by **simpler** and more **scalable** models (with some performance loss).
- Where LLMs differ, is that they are **autonomous**, which allows for **previously impossible** use-cases.

# When you are holding a hammer...

- Suppose you are given the following task, with appropriate resources from management.

## Use case

Find all reviews that concern competitors of our product, and tell us what percentage of them mention it.

# When you are holding a hammer...

Our problem could be decomposed in three parts

- 1 Discovery: Find pages containing reviews.
- 2 Filtering: Find the HTML sections containing the reviews.
- 3 Analysis: Find which reviews reference the product.

Which of these must necessarily be solved with LLMs?  
Which can be solved with traditional ways? What  
trade-offs are we making in the latter case?

... you treat everything as if it were a nail

## The easy solution

We instruct an LLM to retrieve reviews via RAG. We isolate the reviews using prompting, then instruct the LLM to find whether our product is mentioned.

## Result

Code is deployed **within the day**. Management is billed **50,000€** in a week.



# But maybe we can do better

## The system solution

We use a traditional scrapper to find relevant reviews. For each page, we instruct the LLM to extract the reviews. We then run

```
reviews.str.contains(product_name).sum()
```

## Result

Code is deployed within **two days**. Management is billed **20€** in a week.

## Section 2

# Tool-Based Reasoning using ReAct

## Subsection 2

# Thought, Action, Observation

# Tool-Calling

## Tool example

```
from smolagents import tool
@tool
def calculator(expr: str) -> float:
    """
    Evaluate a basic arithmetic expression.

    Args:
    expr: The math expression to evaluate, such as "2 + 3 *
           (4 / 2)".
    """
    return eval(expr, {"__builtins__": None}, {})
```

# What is ReAct?

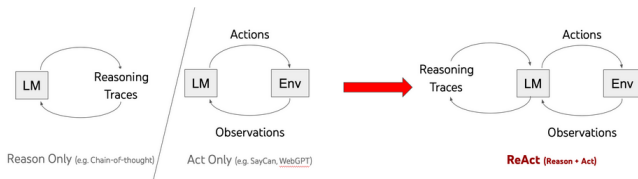


Figure: <https://arxiv.org/abs/2210.03629>

## Definition

**ReAct:** A technique that interleaves reasoning steps (“Thought”) with Actions and Observations to guide an LLM through planning and tool use.

## Tool calling example

```
{  
  "tool_calls": [  
    {  
      "id": "call1",  
      "function": {  
        "name": "calculate",  
        "arguments": {  
          "command": "calculator('2 + 3 * (4 / 2)')"  
        }  
      }  
    }  
  ]  
}
```

>>> 8

## Tool calling chaining example

```
{"tool_calls": [  
  {  
    "id": "stock_call_1",  
    "function": {  
      "name": "get_stock_price",  
      "arguments": {"ticker": "TSLA"}  
    }  
  },  
  {  
    "id": "news_call_2",  
    "function": {  
      "name": "fetch_news_summary",  
      "arguments": {"ticker": "TSLA", "days": 7}  
    }  
  }  
]
```

# High-level Concept

## (1d) ReAct (Reason + Act)

**Thought 1:** I need to search Apple Remote and find the program it was originally designed to interact with.

**Act 1:** `Search[Apple Remote]`

**Obs 1:** The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the `Front Row media center` program ...

**Thought 2:** Apple Remote was originally designed to control the Front Row media center program. I need to search `Front Row` next and find what other device can control it.

**Act 2:** `Search[Front Row]`

**Obs 2:** Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

**Thought 3:** Front Row is not found. I need to search `Front Row (software)` .

**Act 3:** `Search[Front Row (software)]`

**Obs 3:** Front Row is a discontinued media center software ...

**Thought 4:** `Front Row (software)` is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

**Act 4:** `Finish[keyboard function keys]`



## ReAct loop

- Create a plan.
- Identify action using reasoning (CoT).
- Take action.
- Observe result and update plan.

Figure: <https://arxiv.org/abs/2210.03629>

# Early examples

You are an agent that answers questions by using two actions:

- search[query]: Google search the query. You can also use it to calculate math.
- finish[answer]: return the answer

Your generation should have the following format:

**Thought:** to analyze the observation and inform the action.

**Action:** your action

**Question:** If I have seven trillion dollars today, can I buy Apple, Nvidia, and Microsoft? If not, how much more money do I need?

**Figure:**

<https://rdi.berkeley.edu/llm-agents/f24>

**Thought:** I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.

**Action:** search["current market capitalization of Apple, Nvidia, and Microsoft"]

**Observation:** "Nvidia's market cap stood at \$1.832 trillion as of about 10 a.m. ET Wednesday, while Alphabet's was at \$1.814 trillion, bringing Nvidia behind just Microsoft (MSFT) at \$3.015 trillion and Apple (AAPL) at \$2.836 trillion."

**Thought:** Now that I have the market cap for all companies, let's add them together and see if \$7 trillion is enough.

**Action:** search["1.832 trillion + 3.015 trillion + 2.836 trillion"]

**Observation:** 7683000000000

**Thought:** The amount of money needed in addition to the \$7 trillion is \$683 billion.

**Action:** finish["You need an additional \$683 billion to be able to buy Apple, Nvidia, and Microsoft."] ✓

**Figure:**

<https://rdi.berkeley.edu/llm-agents/f24>



## Section 2

# Tool-Based Reasoning using ReAct

## Subsection 3

How much autonomy should we give the agent?

# Agency levels

| Agency Level | Description                                                     | Short name       | Example Code                                                  |
|--------------|-----------------------------------------------------------------|------------------|---------------------------------------------------------------|
| ☆☆☆          | LLM output has no impact on program flow                        | Simple processor | <code>process_llm_output(llm_response)</code>                 |
| ☆☆☆          | LLM output controls an if/else switch                           | Router           | <code>if llm_decision(): path_a() else: path_b()</code>       |
| ★★☆          | LLM output controls function execution                          | Tool call        | <code>run_function(llm_chosen_tool, llm_chosen_args)</code>   |
| ★★☆          | LLM output controls iteration and program continuation          | Multi-step Agent | <code>while llm_should_continue(): execute_next_step()</code> |
| ★★★          | One agentic workflow can start another agentic workflow         | Multi-Agent      | <code>if llm_trigger(): execute_agent()</code>                |
| ★★★          | LLM acts in code, can define its own tools / start other agents | Code Agents      | <code>def custom_tool(args): ...</code>                       |

Figure: [https://huggingface.co/docs/smolagents/en/conceptual\\_guides/intro\\_agents](https://huggingface.co/docs/smolagents/en/conceptual_guides/intro_agents)

## Section 3

### Agent-to-Agent Interaction

#### Subsection 1

#### Single-Agent vs. Multi-Agent

## Section 3

# Agent-to-Agent Interaction

## Subsection 2

# Orchestrating Interaction

## Section 3

# Agent-to-Agent Interaction

## Subsection 3

What do we actually need?